

Distributed Engrammics: Gradient-Free Transfer of a Persistent Fast-Weight Associative State Between Agents

Jacques Gariépy

<https://github.com/JacquesGariépy/engrammics>

June 21, 2026

Abstract

Fast weight programmers treat the fast-weight matrix as an *ephemeral* memory, reset every sequence and private to one network. We study the opposite: this matrix F as a *persistent, transferable* object exchanged between agents, with four operations (inscribe, read, transfer, forget) and a subspace *governance* rule, tested through six pre-specified falsifiable hypotheses. In a controlled associative-memory backend (60 seeds, paired-bootstrap 95% CIs, Holm–Bonferroni), a low-rank state delta extracted from one agent transfers a skill to another *without any gradient step*, separating from both a no-transfer and a norm-matched random control by ≈ 88 points, with a rank dose-response, preserved host skill, projection-based forgetting, and subspace-gated consent. We then run the *identical* protocol on real DeltaNet language-model backends (1.3B and 2.7B, via **flash-linear-attention**). The primary claim holds: a gradient-free engram added to a recipient’s recurrent state transfers the skill (+54 to +66 points over both controls across the two sizes, $p < 0.001$), with a monotone rank dose-response, and subspace governance succeeds once the authorized subspace is read from the model’s *actual* keys (+62 points at 1.3B). Two idealized properties degrade: superposition interferes with the host skill, and subtraction-forgetting damages it. We trace this to shared key directions and confirm it by making the skills use disjoint symbols. Beyond memorized lookups, the fast state can carry a generalizing rule: a vowel/consonant classifier engram, injected into another instance, classifies *held-out* letters above no-transfer, random, and label-swapped controls on DeltaNet, and largely replicates on a second architecture (RWKV-7, where it clearly beats the no-transfer and label-specificity controls). On RWKV-7-2.9B, which can induce a novel rule in context, the engram of a freshly learned Caesar shift transfers that *non-trivial induced* rule: a neutral recipient applies it to held-out letters at 0.44 versus 0.005 (random) and 0.08 (wrong-shift), $p < 10^{-4}$. Transfer also extends beyond two copies of one model: between two genuinely distinct RWKV-7 checkpoints, naive injection fails at chance while a linear key alignment recovers it to the recipient’s own ceiling ($0.04 \rightarrow 0.97$)—a closed-form, skill-independent map fit once per pair with both models frozen, so the per-skill transfer stays gradient-free. We position the method against activation-space task and function vectors and state-soup interpolation, and release a single reproducible harness whose backends are judged by identical criteria.

1 Introduction

A weight matrix is a memory, and a network can learn to program it. Schmidhuber’s fast-weight memories [1] replace recurrent storage by a slow network that emits context-dependent changes to the weights of a fast network; Schlag, Irie and Schmidhuber later showed that linear transformers are precisely fast weight programmers [4]. Across this lineage, from deblurring memories [2] to

attending to the recent past [3] to self-referential weight matrices [5] and modern linear-attention architectures [6], two assumptions are never questioned: the fast-weight matrix is (i) *ephemeral*, reset at the end of each sequence, and (ii) *intra-network*, internal and private to one model.

This paper studies what happens when both assumptions are lifted. We treat the fast-weight matrix F as a persistent, mobile object that can be exchanged between agents. The novelty lies in their combination rather than in “fast weights,” which are established: *persistence*, *peer-to-peer exchange of engrams*, *algebraic governance*, and *targeted forgetting in the fast substrate*. A society of agents that exchanges engrams (low-rank weight deltas) communicates not through tokens or gradients but by writing compact memory traces directly into a peer’s fast state.

Scope. The mechanism applies only to architectures with a single additive recurrent fast-weight state—linear-attention models such as DeltaNet and RWKV-7, which we test. A standard softmax transformer has no such state (its analogue, the KV cache, is a growing per-token buffer, not a fixed-size additive matrix), so the method as stated does not transfer skills into today’s mainstream deployed models. This is the dominant practical limitation, and we state it up front rather than only in the discussion.

Contributions. (1) We define the engrammics framework and its four operations, and we make explicit the shared key-space assumption on which transfer rests (§3–4). (2) We give a falsifiable experimental protocol with pre-specified hypotheses, negative controls, paired-bootstrap confidence intervals and multiplicity correction (§5). (3) We report measured results in a controlled regime: gradient-free transfer separates from both controls and from chance with intervals excluding zero, with a rank dose-response, non-interference, targeted forgetting, governance, and recovery under encoder mismatch via learned alignment (§6). (4) We position the approach against federated learning, distillation, and model merging, reframe “the right to be forgotten” as a verifiable operation in the fast substrate rather than complete erasure, and provide a threat model (§7–8). (5) We release a single reproducible harness whose two backends are judged by identical criteria, and we *execute* the language-model test on real DeltaNet checkpoints (§6.6): the primary gradient-free transfer claim, the rank dose-response, and subspace governance (with a captured-key authorized subspace) are all supported in the DeltaNet backend, while two of the four operations—non-interference and clean forgetting—do *not* work naively and are recoverable only as lossy, tunable trade-offs (§6.6). We trace the failure to key-subspace overlap, a precondition we confirm by construction. (6) We show the transferred engram carries a *generalizing rule*, not merely a memorized table (a vowel/consonant classifier applied to held-out inputs, beating the no-transfer and label-specificity controls on two model sizes and on RWKV-7), and—our most novel empirical result—that an engram transfers between two *distinct* checkpoints once a linear key alignment is learned, where naive injection is at chance (§6.7); we position this against activation-space task/function vectors and state-soup interpolation.

2 Background and related work

Fast weights and fast weight programmers. A linear-attention layer maintains a state S that is written by outer products of keys and values and read by queries [4, 3]. The delta rule [4] replaces the pure Hebbian write by an error-correcting one, which reduces interference; this is the native update of DeltaNet. Gated Delta Networks add an adaptive erase gate over the delta rule [6], and RWKV-7 generalizes the delta rule with vector-valued state evolution [7]. Self-referential

weight matrices let the fast weights modify their own update [5]. In all of these the state is reset per sequence and never leaves the network.

Knowledge transfer between models. Federated learning communicates and averages the *slow* weights of an entire model [8]. Distillation transfers behaviour through outputs. Task arithmetic and model merging add *slow* adapters or task vectors [9], of which LoRA is a common parameterization [10]. Engrammics differs in what circulates: the *fast* weights, as compact composable traces that are semantically localized, added without a gradient step. Associative-memory perspectives on attention [19] are complementary; we use a delta-rule associative memory as the controlled backend. The multi-agent reading draws on classical distributed AI [20] and stigmergy [21].

Vector-based steering and task vectors. A growing line of work distills in-context demonstrations into a single vector that steers a frozen model. Task vectors [11] and function vectors [12] show that in-context behaviour can be compressed into a residual-stream or attention-head vector and re-injected to trigger a task; activation addition [13], representation engineering [14], and in-context vectors [15] likewise add or shift *activation*-space vectors to control one model’s outputs. All operate on transient, per-forward-pass *activations* of a single model. Closest to us in substrate, state soup [16] and document souping [17] linearly mix the recurrent *states* of a state-space model, and cache steering [18] edits a transformer KV cache. Engrammics differs in three ways: the transferred object is a persistent recurrent *fast-weight* state (a DeltaNet delta-rule matrix), not an activation; it is added gradient-free *peer-to-peer* into a separate model instance’s running state rather than steering one model; and it carries an algebra for forgetting (subtraction/projection) and governance (admission within an authorized key subspace). The nearest prior work, state soup, interpolates states *within one instance* and defines no forgetting or governance. We are candid about where our novelty is thin and where it is real: when donor and recipient are two copies of one frozen checkpoint, injecting a state into instance B is close to what state soup already does, so for that case the new content is the *algebra* (governance and forgetting), not the act of moving a state. The genuinely new transfer result is between two *distinct* checkpoints via a learned key alignment (§6.7), which state soup does not address.

3 The engrammics framework

Definition 1 (Engram). An engram is a memory trace written into the fast-weight matrix F as a sum of outer products. A skill engram comprising N associations has rank at most N and may be truncated to rank $r \leq N$.

The framework rests on four operations and one admission rule. *Inscribe*: a slow controller (the *conatus*) emits keys, values and write strengths, and the delta rule writes them into F . *Read*: behaviour is produced by associative readout from F . *Transfer*: an agent emits a low-rank delta ΔF ; a peer integrates it additively, $F \leftarrow F + \alpha \Delta F$, with no gradient step. *Forget*: a targeted trace is removed from F , either by subtracting its engram or by projecting out the subspace it occupies. *Govern*: an incoming engram is admitted only within a receiver-authorized key subspace U , $\Delta F \mapsto UU^\top \Delta F$, passing to the extent it lies in U .

Definition 2 (Agent). An agent is a computational instance comprising a slow controller θ and a persistent fast state F . The controller θ defines how the agent encodes inputs, reads F , writes to F , and governs incoming engrams. In the prototype a minimal agent is the pair (W_k, F) ; in the

language-model backend an agent is a model instance with its per-layer fast recurrent state and its admission rules. Compactly, an agent is $\theta + F$ together with the read/write/transfer/forget/govern operations.

Scope of “agent.” We do not use “agent” in the strong sense of an autonomous, tool-using system. We use it in the minimal sense above: a carrier of fast state F able to read, write, receive or refuse engrams. Autonomy, tools and planning are orthogonal to the mechanism tested here.

Shared key-space assumption. Transfer is meaningful only if the two agents address F in the same key space, i.e. they share the key encoder. This holds by construction when the agents are two instances of the same checkpoint, which is the regime of the language-model test. For genuinely heterogeneous agents it fails, and we study a remedy in §6.4.

4 Operations and formalism

Let keys and queries $k, q \in \mathbb{R}^{d_k}$, values $v \in \mathbb{R}^{d_v}$, and the fast-weight state $S \in \mathbb{R}^{d_k \times d_v}$. The read is

$$o = S^\top q \in \mathbb{R}^{d_v}. \quad (1)$$

The delta-rule write, with prediction $\hat{v} = S^\top k$, correction $c = v - \hat{v}$, and write strength β , is

$$S \leftarrow S + \beta k c^\top = (I - \beta k k^\top) S + \beta k v^\top. \quad (2)$$

We emphasize the operand order: for $S \in \mathbb{R}^{d_k \times d_v}$ the factor $(I - \beta k k^\top)$ acts *on the left* of S . (An earlier draft printed $S(I - \beta k k^\top)$, which is ill-typed for this shape; the implementation has always used (2).)

Engram and transfer. Writing a skill from the zero state yields its engram ΔF . A peer holding state S_B integrates a rank- r truncation,

$$S_B \leftarrow S_B + \alpha \text{trunc}_r(\Delta F), \quad \text{trunc}_r(M) = \sum_{i=1}^r \sigma_i u_i w_i^\top, \quad (3)$$

where $\sigma_i u_i w_i^\top$ are the leading singular triplets of M . The communication cost is $O(r(d_k + d_v))$ floats, which improves on transmitting raw demonstrations only when the skill is compressible to low rank (§6).

Forgetting (two named operations). We distinguish two operations and report each separately. *Forget-by-projection* (exact, requires the keys $K \in \mathbb{R}^{N \times d_k}$):

$$S \leftarrow S - K^\top (K K^\top)^{-1} K S, \quad (4)$$

which nulls the readout for any query in the span of K while preserving the orthogonal complement. *Forget-by-subtraction* (general, key-free): $S \leftarrow S - \Delta F$, used when the keys are not exposed, as in a language-model state; it is exact only up to the nonlinearity of the write order.

Governance (consent as an authorized key subspace). The receiver’s consent policy is an authorized key subspace $U \in \mathbb{R}^{d_k \times m}$; an incoming engram is admitted only as $UU^\top \Delta F$, so it passes to the extent it lies in U . We use a single definition of U for both backends: the span of the skill’s *actual keys* (the d_k side). The prototype reads them from the exact key encoder; the language-model backend captures them from the DeltaNet key short-convolution at write time (its output spans the addressing keys, the subsequent ℓ_2 -norm only rescaling them). An earlier version estimated U from the engram’s leading singular subspace instead; that conflates the key and value sides and fails in the DeltaNet backend (§6.6), so we adopt the captured-key definition throughout. Consent bounds the write *subspace*, not the *content* within it (§8).

The conatus. The slow controller maps content to keys and write strengths. In the controlled backend it is a linear key encoder trained to inscribe non-interfering engrams (near-orthonormal keys); in the language-model backend the pretrained model itself plays this role, since reading in-context demonstrations writes the engram automatically.

5 Experimental protocol

Two backends, one judge. The *toy* backend is a delta-rule associative memory ($d_k = 64$, gaussian content), used to validate the harness and to prove the algebra. The *LM* backend is a pure DeltaNet checkpoint via **flash-linear-attention**, whose per-layer recurrent state is F ; the skill is an in-context injective letter→digit map, and a query is answered by greedy decoding from an injected state. Both backends expose the same operations and are evaluated by the same statistics and verdict function.

Conditions. Per seed we draw two skills X, Y . We measure: the clean-write ceiling (write X alone, read X); a no-transfer control (recipient holds only Y , read X); a norm-matched random-engram control; transfer at ranks $r \in \{1, 2, 4, 8\}$ (the language-model backend adds $r=16$) and at full rank; the recipient’s recall of Y before and after integrating X ; a joint state that reads X then Y and the result of subtracting X ’s engram from it; and integration of X through the admitted versus the denied subspace.

Statistics. Each condition is summarized by its mean and a 95% percentile-bootstrap confidence interval over seeds. Hypotheses are tested by paired bootstrap of the relevant difference; the primary one-sided tests are corrected for multiplicity by Holm–Bonferroni at $\alpha = 0.05$. This correction is applied *within* each backend’s primary family, not across the whole paper; the headline effects (+54 to +66 points) are large enough that an article-wide correction would not change their verdict, but a marginal comparison such as the RWKV-7 rule transfer against the random control ($p = 0.05$, §6.7) would not survive an article-wide family, and we do not claim it does. Decoding is greedy and all seeds are fixed, so runs are reproducible up to CUDA kernel nondeterminism.

Pre-specified hypotheses and decision rules.

Hypothesis 1 (Specific transfer, primary). Full-rank transfer beats both the no-transfer and the random-engram controls. *Rule:* the 95% CI lower bound of each paired difference exceeds 0, and the transfer mean exceeds chance by at least 15 points. If the clean-write ceiling is itself at chance, the task is unlearnable and the outcome is INCONCLUSIVE, not a refutation.

Hypothesis 2 (Non-interference). Integrating X does not destroy Y . *Rule (equivalence)*: the CI lower bound of (recall _{Y} after – before) exceeds -0.10 .

Hypothesis 3 (Targeted forgetting). Subtracting X ’s engram drops recall _{X} while preserving recall _{Y} . *Rule*: CI lower bound of the drop > 0 and of the Y -preservation > -0.10 .

Hypothesis 4 (Governance). An engram is recovered when the authorized key subspace covers its keys and blocked when the policy is orthogonal to them. *Rule*: CI lower bound of (covering policy – orthogonal policy) > 0 .

Hypothesis 5 (Rank dose-response). Recall _{X} increases with the transferred rank. *Rule*: CI lower bound of (max-rank – min-rank) > 0 and the per-rank means are non-decreasing.

Hypothesis 6 (Shared key space). Transfer requires a shared key encoder; under mismatch a learned linear alignment restores it. *Rule*: naive cross-encoder transfer at chance, aligned transfer significantly above it.

6 Results

We first report the controlled regime, where the algebra can be established with full statistical rigour, and then report the same protocol executed on a real DeltaNet language model (§6.6).

6.1 Controlled regime: condition means

Table 1 reports condition means over 60 seeds with 95% bootstrap CIs. Both negative controls sit at chance: the no-transfer control and the norm-matched random engram are indistinguishable from the 12.5% chance level, so any transfer effect is specific rather than an artifact of adding mass to the state.

Forget-by-projection. The exact projection variant, evaluated on the associative-memory backend, drives the target skill’s read signal to 0.00 ± 0.00 while preserving the other skill at $100\% \pm 0$ when their key subspaces are disjoint, and degrades gracefully as the subspaces coincide (§6, sensitivity).

6.2 Hypothesis tests

Table 2 reports the pre-specified tests. The primary hypothesis is supported: full transfer beats the no-transfer control by +87.5 points and the random-engram control by +87.9 points, each 95% CI excluding zero with $p < 0.001$ surviving Holm correction. Recall grows monotonically with rank (H5), the recipient’s Y is preserved (H2), subtraction-forgetting drives recall _{X} to floor while Y is retained (H3), and the consented key subspace gates integration (H4).

6.3 Capacity and the conatus

The sharpness of transfer and forgetting is governed by the ratio of capacity to load. Training the conatus to inscribe non-interfering engrams raises recall as the number of stored associations grows (Table 3), from 84.6% to 97.2% at 32 associations.

Table 1: Condition means with 95% bootstrap CIs, controlled regime, 60 seeds. Chance = 12.5%.

Condition	Recall (%)	95% CI
Clean-write ceiling	100.0	[100.0, 100.0]
No-transfer control	12.5	[9.8, 15.2]
Random-engram control	12.1	[9.0, 15.2]
Transfer, rank 1	28.7	[26.0, 31.7]
Transfer, rank 2	55.0	[51.5, 58.5]
Transfer, rank 4	88.8	[85.8, 91.5]
Transfer, rank 8	100.0	[100.0, 100.0]
Transfer, full rank	100.0	[100.0, 100.0]
Y before adding X	100.0	[100.0, 100.0]
Y after adding X	100.0	[100.0, 100.0]
Joint state, read X	100.0	[100.0, 100.0]
Joint state, read Y	100.0	[100.0, 100.0]
After subtraction-forgetting X , read X	0.2	[0.0, 0.6]
After subtraction-forgetting X , read Y	100.0	[100.0, 100.0]
Admitted subspace, read X	100.0	[100.0, 100.0]
Denied subspace, read X	12.5	[9.8, 15.2]

6.4 Heterogeneous key spaces and alignment

Transfer presupposes a shared key encoder. Table 4 confirms the failure mode and its remedy: when two agents use different encoders, naive transfer collapses to chance ($12.5\% \pm 14.3$), because the received delta is addressed in the wrong space; a linear alignment learned from 128 shared anchors—a closed-form least-squares map, fit from anchors rather than the transferred skill—fully restores it to the shared-encoder upper bound (the gradient-free precision of §6.7 applies here too).

6.5 Sensitivity

Integration factor. At rank 4, recall of the transferred skill rises with the integration factor α , from $65.6\% \pm 17.6$ at $\alpha = 0.25$ to $90.0\% \pm 9.4$ at $\alpha = 1.0$ and $91.9\% \pm 8.2$ at $\alpha \geq 1.5$, while the recipient’s pre-existing skill is preserved at 100% throughout.

Subspace overlap. Forgetting by projection preserves the other skill at 100% across partial key-subspace overlaps and collapses only when the subspaces nearly coincide ($16.9\% \pm 14.9$ at full overlap). Clean forgetting is therefore conditioned on the engrams occupying distinct subspaces.

Noise on the transmitted engram. Recall is robust to gaussian corruption of the transmitted rank-8 engram up to $\sigma = 0.4$ (100%). Random noise is thus tolerated; the relevant threat is structured adversarial writing rather than noise (§8).

6.6 Language-model results

We now run the *same* pre-specified protocol on a real language model. The backend is the pure DeltaNet checkpoint `fla-hub/delta_net-1.3B-100B` (1.3B parameters, 24 layers, 16 heads, head

Table 2: Pre-specified hypothesis tests, paired bootstrap, 95% CI, Holm-corrected primary tests. Δ in accuracy points.

	Test	Δ	95% CI	Verdict
H1a	full > no-transfer	+87.5	[84.8, 90.2]	SUPPORTED
H1b	full > random	+87.9	[84.8, 90.8]	SUPPORTED
H4	admit > deny	+87.5	[84.8, 90.2]	SUPPORTED
H5	rank 8 > rank 1	+71.2	[68.3, 74.0]	SUPPORTED
H2	Y after – before	+0.0	[0.0, 0.0]	SUPPORTED
H3	subtraction-forget: drop on X	+99.8	[99.4, 100.0]	SUPPORTED
H3	subtraction-forget: keep Y	+0.0	[0.0, 0.0]	SUPPORTED

Table 3: Mean recall versus number of stored associations N , before and after training the conatus (associative-memory backend, 120 skills per cell).

N	Untrained (%)	Trained (%)
8	99.7	100.0
16	97.6	99.9
24	92.1	99.1
32	84.6	97.2

dimension 128, trained on 100B tokens), loaded in `bfloat16` on a single NVIDIA RTX 3090 via `flash-linear-attention` 0.5.1 and `transformers` 5.12.1, over 30 seeds with greedy decoding. The skill is a random injective letter→digit map of 5 pairs (chance = 10%).

What is transferred, and how. The fast state F we move is the *per-layer recurrent state*, the DeltaNet associative memory; this is the only component that superposes additively. DeltaNet also keeps a short-convolution window over recent tokens, which is local and not additively transferable, so reading an injected recurrent state requires a realistic convolution window: we supply one with a constant, skill-agnostic primer whose own recurrent write is overwritten by the injected engram and whose keys lie outside the task alphabet, so it carries no skill information and is identical across all conditions. The skill table is read three times when writing the engram: one pass is too weak for this checkpoint to store five novel associations, whereas after three passes the natural in-context recall ceiling is 100% (Table 5, clean-write). Each fresh sequence receives a single beginning-of-sequence token, so a continuation never injects a spurious one mid-stream. These are presentation choices for a base (non-instruction-tuned) checkpoint; they do not alter the engram algebra, which is identical to the controlled backend.

Specificity and the role of repetition. Two checks guard against weaker readings of the transfer. First, the three-pass presentation is not a fragile “prompt rehearsal” tuned to a favorable count (15 seeds per setting): a single pass writes too weak an engram (clean-write ceiling 30.7%, transfer 12.0%, which is INCONCLUSIVE against the no-transfer control, $p = 0.12$), but *two* passes already saturate it (ceiling 100%, transfer 70.7%, $p < 0.001$) and three are equivalent (transfer 68.0%), so the effect is stable for any reps ≥ 2 and not tuned to a single count. Second, beyond the norm-matched random engram we add two *structured* negative controls (30 seeds): a *shuffled-values* engram (the donor’s exact keys with its values permuted) and a *wrong-skill* engram (an unrelated

Table 4: Cross-agent transfer under encoder mismatch, 20 seeds (mean $\pm$ s.d.).

Setting	Recall of transferred skill (%)
Shared encoder (upper bound)	100.0 $\pm$ 0.0
Different encoders, naive transfer	12.5 $\pm$ 14.3
Different encoders, learned alignment	100.0 $\pm$ 0.0

skill). Real transfer (74.0%) beats the shuffled-values control (20.0%; $\Delta = +54.0$, [47.3, 60.7]) and the wrong-skill control (5.3%; $\Delta = +68.7$), each $p < 0.0001$. The shuffled-values control sits above chance (the right keys do address the state), but the recovered behavior is specific to the engram’s *content*, not its structure, spectrum, or norm.

Multi-token associations and a key-pairing control. The recall above maps each key to a single token. To check that the engram carries genuine multi-token associations, we move to a letter $\rightarrow$ two-digit task, scoring a value correct only when *both* decoded tokens match (chance $\approx 1/90$ per pair). Transfer survives: injecting the donor engram into a recipient holding an unrelated skill recovers 63.5% (40 seeds) against a 98.5% clean ceiling, versus 1.0% (no-transfer) and 0.0% (non-matched random). We also add the harder structural control the reviewer-minded reader expects, a *shuffled-key* engram (the donor’s exact keys and values, but with the key $\leftrightarrow$ value pairing permuted): it sits at 10.0%, and real transfer beats it by +0.535 ($p < 10^{-4}$). So the engram transports the specific key $\rightarrow$ value *pairing*, not merely the presence of the right keys and values, and it does so for multi-token outputs.

Condition means. Table 5 reports the 30-seed condition means. The clean-write ceiling is 100%, so the task is learnable and the run is not vacuous; both negative controls sit at or below the 10% chance level.

Hypothesis tests. Table 6 applies the identical verdict function. The **primary hypothesis H1 is supported in this real DeltaNet backend**: full transfer recovers the donor’s skill at 68.7%, beating the no-transfer control by +61.3 points and the random-engram control by +66.0 points, each 95% CI excluding zero with $p < 0.001$ after Holm correction, and exceeding chance by far more than the 15-point margin. The rank dose-response (H5) is also supported and monotone. Unlike the controlled regime, where the skill is exactly rank 5, here it requires rank ≥ 8 (rank 4 is still near chance), because the model’s keys are not orthogonal and the skill spreads over many singular directions. **Governance (H4) is likewise supported**: admitting the engram through the skill’s authorized key subspace recovers X at 69.3%, essentially the full-transfer level, while the orthogonal complement leaves it at the 7.3% chance floor ($\Delta = +62.0$, [56.0, 68.0], Holm-corrected). This holds only with the captured-key definition of the authorized subspace (§4); the earlier engram-SVD estimate gave the opposite, sub-chance ordering, because five singular directions of the state do not coincide with the skill’s key subspace.

What degrades, and why. Two idealized properties weaken on the dense, non-orthogonal LM state, and both for the same reason: the two skills do not occupy *disjoint* key subspaces, whereas in the controlled regime they do by construction. *Non-interference (H2) fails*: superposing X onto Y lowers Y recall from 100% to 74%, the crosstalk of adding outer-product mass in a shared 128×128 state whose key directions overlap. *Targeted forgetting (H3) is asymmetric*: subtracting X ’s engram

Table 5: Language-model condition means with 95% bootstrap CIs, DeltaNet-1.3B, 30 seeds. Chance = 10.0%.

Condition	Recall (%)	95% CI
Clean-write ceiling	100.0	[100.0, 100.0]
No-transfer control	7.3	[3.3, 11.3]
Random-engram control	2.7	[0.7, 5.3]
Transfer, rank 1	7.3	[4.0, 10.7]
Transfer, rank 2	7.3	[3.3, 11.3]
Transfer, rank 4	14.7	[9.3, 20.0]
Transfer, rank 8	44.0	[36.0, 52.0]
Transfer, rank 16	68.7	[61.3, 75.3]
Transfer, full rank	68.7	[61.3, 75.3]
Y before adding X	100.0	[100.0, 100.0]
Y after adding X	74.0	[68.0, 80.0]
Joint state, read X	50.7	[44.7, 56.7]
Joint state, read Y	92.7	[88.0, 96.7]
After subtraction-forgetting X , read X	2.0	[0.0, 4.0]
After subtraction-forgetting X , read Y	21.3	[14.7, 28.7]
Admitted subspace, read X	69.3	[62.7, 76.0]
Denied subspace, read X	7.3	[3.3, 11.3]

drives recall_X to floor ($50.7\% \rightarrow 2.0\%$, drop supported) but also collapses Y ($92.7\% \rightarrow 21.3\%$), because writing Y on top of a state that already holds X entangles the two through the delta rule’s order-dependent write. This is the “exact only up to the nonlinearity of the write order” caveat of (4), confirmed empirically. Replacing subtraction by key-projection forgetting (using the captured keys) removes X just as cleanly but still does not rescue Y , because the two skills share the format keys (“=”, “;”) that every recall query passes through. Both failures are thus traceable to key-subspace overlap, a precondition stated in the theory, rather than to the transfer mechanism itself.

Confirming the precondition. To check that key-subspace overlap *causes* the two failures, rather than merely correlating, we vary the overlap in three controlled steps (16 seeds, key-projection forgetting). (i) OVERLAP: both skills draw from the same alphabet $\text{ABCDEFGH} \times 0\text{--}9$ with shared separators “=”, “;”. (ii) SYMBOLS: disjoint symbols (X over $\text{ABCD} \times 0\text{--}4$, Y over $\text{EFGH} \times 5\text{--}9$) but still shared separators. (iii) FULL: disjoint symbols *and* disjoint separators (X uses “=”, “;”; Y uses “:”, “|”), so the two skills share no key directions at all. Table 7 shows a clean monotone progression. Non-interference (H2) reaches the noise floor as soon as the *symbols* are disjoint ($\Delta_Y : -0.20 \rightarrow 0.00$), even with shared separators, so H2’s failure is fully explained by symbol-key overlap. Forgetting improves progressively and only becomes near-surgical at FULL disjointness: the drop on X rises $0.56 \rightarrow 0.98 \rightarrow 1.00$ while the collateral damage to Y falls $-0.61 \rightarrow -0.56 \rightarrow -0.30$. The residual -0.30 at FULL indicates that the shared format still couples the skills slightly through the model’s positional processing; we do not claim fully surgical forgetting. This dose-response has a clean endpoint for interference and confirms that the two degradations track key overlap, not the transfer mechanism.

Table 6: Language-model pre-specified hypothesis tests, DeltaNet-1.3B, 30 seeds, paired bootstrap, 95% CI, Holm-corrected primary tests. Δ in accuracy points.

	Test	Δ	95% CI	Verdict
H1a	full > no-transfer	+61.3	[55.3, 67.3]	SUPPORTED
H1b	full > random	+66.0	[59.3, 72.7]	SUPPORTED
H4	admit > deny	+62.0	[56.0, 68.0]	SUPPORTED
H5	rank 16 > rank 1	+61.3	[55.3, 67.3]	SUPPORTED
H2	Y after – before	−26.0	[−32.0, −20.0]	NOT SUPPORTED
H3	subtraction-forget: drop on X	+48.7	[42.0, 55.3]	SUPPORTED
H3	subtraction-forget: keep Y	−71.3	[−80.0, −62.0]	NOT SUPPORTED

Table 7: Interference and forgetting versus key-subspace overlap, DeltaNet-1.3B, 16 seeds. Δ_Y = change in Y -recall when X is added (H2); $\text{drop}_X/\text{keep}_Y$ = effect of key-projection forgetting of X on the joint X -then- Y state (H3).

Overlap regime	Δ_Y (H2)	drop_X	keep_Y
OVERLAP (shared symbols & seps)	−0.20	+0.56	−0.61
SYMBOLS (disjoint symbols)	+0.00	+0.98	−0.56
FULL (disjoint symbols & seps)	+0.00	+1.00	−0.30

Engineering non-interference at injection time. The disjoint-symbol result requires control over the skills’ vocabulary, which a recipient may not have. The same governance projector gives a remedy that needs neither disjoint data nor retraining: before adding a donor engram, project it onto the orthogonal complement of the host’s captured key subspace, $\Delta F' = (I - U_Y U_Y^\top) \Delta F$, so the addition has no component in the host’s key directions. On DeltaNet-1.3B (30 seeds), this removes the host damage entirely— Y recall stays at 1.00 ($\Delta_Y = 0.00$, versus the naive -0.33)—at the cost of transferred-skill fidelity, which falls from 0.75 to 0.41 as the part of X that overlaps Y ’s keys is discarded. Non-interference is therefore not lost but *purchasable*: orthogonal injection trades transfer fidelity for host preservation along a tunable curve, with full host preservation at one end. The same projector that gates an engram by consent (H4) thus also allocates it away from collisions.

Engineering Y-safe forgetting. Targeted forgetting (H3) admits the dual remedy. When X and Y share keys, removing X ’s shared component necessarily removes Y ’s use of those keys, so perfect forgetting with perfect preservation is impossible; the controllable choice is to forget only X ’s key directions that are *orthogonal* to Y ’s, $P_{X \setminus Y} = \text{proj}((I - U_Y U_Y^\top) U_X)$. In the hard regime where X and Y share the alphabet (24 seeds), naive subtraction drives recall_X from 0.55 to 0.02 but collapses Y from 0.87 to 0.24 ($\Delta_Y = -0.63$); full key-projection is no better ($\Delta_Y = -0.66$). Forgetting only X ’s orthogonal directions instead *preserves* Y ($0.87 \rightarrow 0.93$, $\Delta_Y = +0.07$) while still removing part of X (recall_X $0.55 \rightarrow 0.33$). Forgetting, like superposition, is thus a tunable trade-off rather than an all-or-nothing operation: one end removes X completely and damages Y , the other preserves Y exactly and forgets X only where it does not collide. Both idealized properties (H2, H3) are recoverable with the same captured-key projector, turning two scale-limitations into controllable operations.

Robustness across model size. The full protocol on the larger `fla-hub/delta_net-2.7B-100B` (2.7B parameters, 32 layers, 20 heads) over 20 seeds reproduces every verdict. The clean ceiling is again 100%; full transfer recovers 61.0%, beating the no-transfer control by +54.0 points ([45.0, 63.0]) and the random control by +60.0 points ([51.0, 68.0]); the rank dose-response is supported and monotone; governance holds with the captured-key subspace (admit 59.0% vs deny 7.0%, $\Delta = +52.0$, [43.0, 61.0]); and the same two degradations recur—non-interference (Y : 100% $\rightarrow$ 71%, -29 points) and the keep- Y leg of subtraction-forgetting (88% $\rightarrow$ 27%, -61 points). All Holm-corrected, $p < 0.001$. Because the support of H1/H4/H5 and the failure of H2/H3-keep replicate across two model sizes, the pattern appears to be a property of the mechanism rather than of a single checkpoint.

6.7 Memory or skill? Transferring a generalizing rule

The recall task of §6.6 transfers a memorized table: every queried key was shown in the donor’s demonstrations, so a skeptic may fairly object that we move a lookup, not a capability. The right test is whether a transferred engram acts correctly on inputs the donor *never demonstrated*. This separates a transferred *dictionary* (only seen keys recovered) from a transferred *rule* (held-out inputs handled).

Arbitrary symbolic rules: a model limit, not a mechanism limit. We first ask whether these checkpoints can learn a *novel* symbolic rule in context at all. We teach a Caesar shift $f(x) = x+k$ on a train set of letters and query *held-out* letters. They cannot: held-out accuracy is at the $1/26$ chance floor for every shift $k \geq 2$ on both 1.3B and 2.7B, and the one shift that appears to work, $k=1$ (successor), is a pretrained prior, since a *wrong*-shift demonstration still elicits the successor 30–40% of the time. Repeating the table *lowers* held-out accuracy (1.3B successor: 0.50 at one pass, 0.12 at three), i.e. repetition drives memorization, not induction. So an arbitrary new rule cannot be transferred on these checkpoints, but the bottleneck is the base model’s inability to *learn* the rule in context, not the engram channel; a model that can learn the rule lets us test the channel directly, which we do on RWKV-7-2.9B below.

A known-concept rule generalizes, and transfers. We therefore use a rule the pretrained model already represents: a vowel/consonant classifier with a per-seed label assignment (vowel $\rightarrow a$, consonant $\rightarrow b$). Both checkpoints apply this concept to held-out letters in context (0.73–0.79 correct vs. a 0.21–0.27 wrong-label baseline). We write its engram from a train set of letters, inject it (recurrent state only, neutral primer) into a recipient holding an unrelated constant-output skill, and score *held-out* letters never demonstrated. Table 8 reports the result, replicated on both sizes. Injected into a neutral recipient the engram carries the rule: it classifies held-out letters at 0.75 (1.3B) / 0.75 (2.7B), while every control sits near zero (no-transfer 0.00, random ≤ 0.03 , and a label-swapped engram 0.01–0.06). Superposed onto the recipient’s existing skill, the rule still transfers above all three controls: +0.13 (1.3B) and +0.17 (2.7B) over no-transfer, and +0.12/+0.10 over the label-swapped control (all $p < 10^{-4}$, 35/52 seeds), which shares the engram’s structure but encodes the opposite assignment. The superposed transfer is bounded well below the ceiling by the same interference as H2 (the recipient’s skill competes), but it is statistically unambiguous and, being measured on held-out inputs against a label-specificity control, is *not* a memorized lookup. We state this carefully: the result shows the transferred fast state can carry *at least one* simple generalizing rule, not only demonstrated key–value pairs; it does not yet establish arbitrary rule transfer or multi-step procedural transfer, which require a model that can learn the rule in context (§6.7, first paragraph).

Table 8: Transfer of a generalizing vowel/consonant rule, scored on *held-out* letters never demonstrated. “Engram alone” injects the rule engram into a neutral recipient; “superposed” adds it to a recipient holding an unrelated skill. Controls: recipient alone (no-transfer), a norm-matched random engram, and a label-swapped engram (same structure, opposite assignment). 95% bootstrap CIs.

Condition (held-out accuracy)	DeltaNet-1.3B (35 seeds)	DeltaNet-2.7B (52 seeds)
Engram alone (rule ceiling)	0.746 [0.671, 0.811]	0.752 [0.700, 0.798]
No-transfer control	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
Random-engram control	0.025 [0.007, 0.050]	0.007 [0.000, 0.022]
Label-swapped control	0.007 [0.000, 0.018]	0.062 [0.034, 0.096]
Superposed transfer	0.132 [0.064, 0.211]	0.166 [0.101, 0.236]
Δ vs. no-transfer	+0.132 ($p < 10^{-4}$)	+0.166 ($p < 10^{-4}$)
Δ vs. random	+0.107 ($p < 10^{-4}$)	+0.159 ($p < 10^{-4}$)
Δ vs. label-swapped	+0.125 ($p < 10^{-4}$)	+0.103 ($p < 10^{-4}$)

A non-trivial induced rule transfers on a rule-capable model. The vowel/consonant rule is one the model already knows; the stronger test is a rule it must *induce* in context. RWKV-7-2.9B can do this where the smaller models cannot: it applies a Caesar shift $f(x) = x+2$ to held-out letters at 0.33–0.44, against a 0.04 chance floor and a 0.08 wrong-shift baseline. We write the engram of this induced rule from a train set of letters and inject it into a neutral recipient, then score held-out letters never demonstrated (50 seeds). The engram carries the rule: the recipient applies $f(x) = x+2$ to held-out letters at 0.440 ([0.385, 0.492]), versus 0.005 for a random engram and 0.080 for an engram of a *different* shift ($\Delta = +0.435$ and $+0.360$ respectively, both $p < 10^{-4}$). The wrong-shift control is the decisive one: it shares the engram’s format and structure but encodes a different transformation, and it does not reproduce the target, so what transfers is the specific rule, not a prior or the engram’s shape. This is, on a model able to learn the rule in context, gradient-free transfer of a *non-trivial, in-context-induced* generalizing rule to a recipient, applied to inputs the donor never showed. We tested a single transformation (one Caesar shift on one model), so this is a proof of possibility, not evidence that any rule transfers; the same RWKV-7 implementation caveat applies, and the rule’s in-context ceiling (≈ 0.44) bounds what any transfer can recover.

A constant output style also transfers. A simpler generalizing behavior corroborates this. A *constant output style*—“always answer with token *c*”—applies to every key by construction, so held-out keys again test generalization. Over 40 seeds the style engram injected into a recipient with a different style makes held-out queries emit *c* at 35.3% ([23.4, 47.5]) versus 0.0% (no-transfer) and 0.3% (random), $\Delta = +35.3/+35.0$, $p < 10^{-4}$. The behavior transfers to inputs the donor never showed, again bounded below ceiling by interference.

Replication on a second architecture. The rule transfer is not specific to DeltaNet. We repeat it on RWKV-7 [7], a different linear-attention model whose generalized delta rule produces a per-layer recurrent state of the same shape; our harness reads and writes that state without modification. The base RWKV-7-1.5B applies the vowel/consonant rule to held-out letters in context as well as DeltaNet does (0.74–0.78). Transferring its rule engram (52 seeds) gives a clean ceiling of 0.80 and

a superposed transfer of 0.147, clearly above the no-transfer control (0.00; $\Delta = +0.147$, $p < 10^{-4}$) and the decisive label-swapped specificity control (0.03; $\Delta = +0.113$, $p = 0.0004$). Against the norm-matched random engram the separation is weaker: the random control is itself elevated to 0.08 on this two-class task, and the gap ($\Delta = +0.067$) is only borderline ($p = 0.05$, CI $[-0.01, 0.15]$). So the rule transfers on RWKV-7 in the sense that matters—it beats both the recipient’s own state and a same-structure wrong-label engram—but its margin over additive noise is thinner than on DeltaNet, and we do not overstate it. We note that this run uses the **flash-linear-attention** RWKV-7 implementation, which its authors flag as not bit-exact against the reference; the effect is a coarse behavioral measure and exact RWKV-7 numbers should be cross-checked against the official code. As with DeltaNet, donor and recipient are two instances of one checkpoint, so this demonstrates architecture-generality, not transfer across distinct checkpoints.

Transfer across distinct checkpoints. The scenario that motivates the framework is exchange between *different* agents, not two copies of one model. We test it with two genuinely distinct RWKV-7-1.5B checkpoints—the reasoning-oriented **g1** and the multilingual-base **world**—which share architecture and dimensions but were trained differently, so their recurrent states are dimensionally compatible but addressed in different key bases. Both recover the recall task from their own engram (1.00). Injecting the donor’s (**g1**) engram directly into the recipient (**world**) fails, at 0.040, i.e. chance: the engram is meaningless in the recipient’s basis, exactly the heterogeneity failure the controlled regime predicts (§6.4). The remedy is the controlled-regime remedy. From 24 shared anchor skills processed by both models we learn a per-layer, per-head linear map W with $F_B \approx W F_A$ (if the recipient’s keys are the donor’s mapped by W , the engram $F = \sum_i k_i v_i^\top$ maps the same way), and transfer $W e_X^A$ instead. This recovers recall to 0.968 ($[0.936, 0.992]$)—the recipient’s own ceiling—a +0.928 gain over naive transfer ($p < 10^{-4}$, 25 seeds).

We are precise about what “gradient-free” means on this path, since it bears on the headline claim. The map W is the *closed-form* ridge least-squares solution $W = X_B X_A^\top (X_A X_A^\top + \varepsilon I)^{-1}$ over the anchors’ state matrices, solved per layer and head by a single matrix inverse: there is no gradient descent, no backpropagation, and no update to either model’s weights—both checkpoints stay frozen and W is a one-shot change of basis between them, not training. Two properties keep this honest. First, W is fit from *generic* shared anchors, not from the transferred skill X , so it is *skill-independent*: the same W aligns any engram between this donor–recipient pair. Second, it is therefore *amortized*—solved once per pair and reused for every subsequent transfer, while each skill transfer remains the same single additive injection $W e_X^A$. The cost of learning the channel is paid once and does not depend on what is sent through it; the per-skill operation is still gradient-free addition. Gradient-free engram transfer therefore extends to distinct checkpoints once this linear key alignment is learned, reproducing at language-model scale the alignment result of the controlled regime (Table 4). The alignment presumes the two agents share architecture and dimensions and admit shared anchors; genuinely heterogeneous agents (different architectures or sizes) remain open.

Reading. The pre-specified claims—gradient-free transfer, rank dose-response, and captured-key subspace governance—hold in the DeltaNet backend at both 1.3B and 2.7B, and what circulates is not only a memorized table: the fast state can carry at least one simple generalizing rule (§6.7), which replicates on a second architecture. What does *not* survive intact is interference-free superposition and surgical forgetting; we localize the cause to shared key directions and confirm it by construction (Table 7). The same harness that supports the algebra in the controlled regime exposes its limits in the DeltaNet backend and points to the missing precondition, disjoint key allocation, for future work.

6.8 What is and is not established

To keep the claims and their boundaries in one place, Table 9 lists what the experiments support and what they do not.

Table 9: Scorecard of claims. “Controlled” = associative-memory backend; “DeltaNet” = the 1.3B/2.7B language-model backend.

Claim	Status	Where
Gradient-free fast-weight transfer	supported	controlled + DeltaNet
Rank dose-response (more rank, more recall)	supported	controlled + DeltaNet
Governance by captured key subspace	supported	controlled + DeltaNet
Replication across model size (1.3/2.7B)	supported	DeltaNet
Multi-token associations + key pairing	supported	DeltaNet
Transfer of one simple generalizing rule	supported	DeltaNet + RWKV-7
Replication on a second architecture	supported (random borderline)	RWKV-7
Transfer of a non-trivial induced rule (Caesar)	supported	RWKV-7-2.9B
Interference-free superposition, naive	not supported	DeltaNet (shared keys)
Non-interference via orthogonal injection	supported (fidelity cost)	DeltaNet
Surgical forgetting, naive	not supported	DeltaNet (shared keys)
Y-safe forgetting via X-only projection	supported (partial removal)	DeltaNet
Multi-step / compositional procedural transfer	not shown	—
Transfer across distinct checkpoints (same arch)	supported (learned alignment)	RWKV-7 g1→world
Transfer across architectures / sizes	not shown	—
Governance for opaque (key-hidden) agents	not shown	—
Adversarial robustness (structured poisoning)	not shown	—

7 Discussion

What is novel. The combination, not the substrate. Federated learning moves slow weights, distillation moves outputs, task arithmetic moves slow adapters; engrammics moves fast weights as compact, composable, locally meaningful traces, integrated without a gradient step, with linear-algebraic governance and forgetting. The controlled results establish that this algebra behaves as claimed, and the language-model run (§6.6) settles the open question in large part: the core transfer, the rank dose-response, and subspace governance (with a captured-key authorized subspace) all survive on a real DeltaNet, while interference-free superposition and surgical forgetting do not, because two skills there share key directions. In short, fast-weight transfer and consent are real in a real DeltaNet backend, while clean non-interference and erasure remain conditioned on disjoint key subspaces.

Cost is conditional. The low-rank cost $O(r(d_k + d_v))$ improves on transmitting raw demonstrations only when the skill is compressible. In our setting a rank-4 engram (448 floats) recovers the skill at 88.8% and is cheaper than raw replay (768 floats), whereas a full rank-8 engram (896 floats) is not. The correct claim is useful compression for low-rank skills, not unconditional sub-linearity.

8 Governance and security

Forgetting is verifiable, not complete. Removing a trace from F by projection (4) or subtraction is auditable and, in the controlled regime, drives the target recall to floor while preserving others. In the DeltaNet backend the removal of the target is just as sharp (recall_X to floor), but it is *not surgical*: subtracting an engram from a state in which a later skill was written on top also damages that later skill (§6.6, H3), because the delta-rule write is order-dependent. Even setting that aside, this is a guarantee *in the fast substrate only*: it does not certify the absence of the information from slow weights, caches, logs, outputs or correlates. Mapped onto data-protection regimes, this is a partial, verifiable erasure of the fast trace, not legal conformance.

Consent bounds the subspace, not the content. Governance admits an engram only within U ; a hostile but in-subspace write still passes. The projection is also only as good as the estimate of U : it works in the DeltaNet backend precisely because we build U from the model’s *actual* keys (§6.6, H4 supported, admit 69% vs deny 7%), whereas estimating U from the engram’s leading singular subspace inverts the test. Subspace governance in this regime therefore presumes access to the receiver’s key projection, and in all cases needs signatures, write quotas and anomaly detection on ΔF beyond the projection.

Threat model. Engram poisoning (a peer emits a corrupted delta; random noise is tolerated up to moderate σ , but structured adversarial writes are untested), state saturation (mass writes beyond capacity induce interference and implicit forgetting), in-subspace malicious writes, and key collisions (skills sharing key directions interfere and forget jointly).

9 An interpretive note

The slow/fast split admits a Spinozan reading: F as the body’s dispositional state, the affections of the body modified by encounters, and the conatus as the persistent effort that governs inscription; associative memory by outer products is prefigured in the propositions on the linkage of simultaneous affections [22]. This framing motivated the design and is required by none of the results.

10 Limitations

What this does not prove. To be unambiguous about scope: we do *not* claim general skill transfer between AI agents, a solution to agent memory, legal “right to be forgotten,” or transfer of broad knowledge. What we measure is narrower and concrete: a fast-weight, associative-memory state can be extracted, moved, and injected into another agent without a gradient step, with an effect that beats negative controls; this survives partially in real linear-attention LMs; two idealized properties (clean superposition, surgical forgetting) hold only as tunable trade-offs; and cross-agent transfer is shown only between same-architecture checkpoints with a learned alignment. The tasks are synthetic (letter/digit recall, a binary classifier, one induced shift), not rich procedures. Read every “skill” below as “associative skill trace in the fast state.”

The controlled results (gaussian content, low load, artificial skills) establish an algebraic property, not a real procedural or semantic skill. Clean forgetting is conditioned on subspace disjointness; transfer is conditioned on a shared key space, with heterogeneity requiring alignment and anchors. Compression is useful only for low-rank skills. Robustness to random noise is shown; adversarial

robustness is not. The language-model test has now been run, but it remains narrow: two pure-DeltaNet checkpoints (1.3B and 2.7B), a synthetic letter→digit recall task, a shared-checkpoint (hence shared-key) regime, and a presentation that shows the table three times to write a usable engram. It establishes that gradient-free fast-weight transfer and captured-key subspace governance survive at this scale, while interference-free superposition and surgical forgetting do not, for want of disjoint key subspaces. The generalizing rules we transfer are a binary classifier (vowel/consonant), a constant output style, and a single Caesar shift; multi-step or compositional procedures are untested. The Caesar result holds only on RWKV-7-2.9B, the one model in our suite that induces such a rule in context: on the smaller checkpoints an arbitrary new rule does not transfer because the base model cannot learn it, so that negative is a model limit, not a property of the engram channel. The RWKV-7 runs use the **flash-linear-attention** implementation, which its authors flag as not bit-exact; the effects are coarse behavioral measures and should be cross-checked against the reference code. Most transfer runs use two instances of one checkpoint, where the shared-key-space assumption is trivially met; we extend to two *distinct* same-architecture checkpoints via a learned linear alignment (§6.7), but transfer across different architectures or sizes is untested. The governance result likewise presumes access to the receiver’s key projection, which holds for two instances of one checkpoint but not for genuinely opaque third-party agents.

11 Conclusion

Treating fast weights as a persistent, transferable substrate makes gradient-free skill transfer, targeted forgetting and subspace governance realizable and measurable in a controlled regime, under an explicit shared key-space assumption, with effects that separate from strong baselines at 95% confidence. Running the identical harness on the fast state of a real DeltaNet language model shows that the decisive property, gradient-free skill transfer through the fast weights, survives in this back-end (+61 to +66 points over both controls, Holm-corrected), as do its rank dose-response and, once the authorized subspace is read from the model’s actual keys, subspace governance (+62 points). Interference-free superposition and surgical forgetting do not survive naively, because skills that share key directions cannot be separated by linear algebra; but both become *controllable* with the same captured-key projector—projecting an incoming engram off the host’s keys preserves the host exactly at a transfer-fidelity cost, and forgetting only a skill’s host-orthogonal directions is host-safe while removing it partially. They are thus tunable trade-offs with a safe endpoint, not flat failures. On the question of what *kind* of thing transfers, the engram carries more than a memorized table: a generalizing classification rule (vowel/consonant) transfers to held-out inputs above no-transfer, random, and label-swapped controls on two DeltaNet sizes, and replicates on RWKV-7 against the no-transfer and label-specificity controls (its margin over additive noise there is only borderline), and on RWKV-7-2.9B the engram of a freshly induced Caesar shift transfers that non-trivial rule to held-out inputs above random and wrong-shift controls. Transfer also extends beyond two copies of one model: between two distinct same-architecture RWKV-7 checkpoints, naive injection is at chance while a learned linear key alignment recovers the recipient’s own ceiling. What remains untested is multi-step or compositional procedural transfer, transfer across *different architectures or sizes*, governance against opaque agents, and adversarial robustness. The mechanism is not confined to memorized lookup, and it is not yet a universal cross-agent protocol; how far it scales to richer skills and harder heterogeneity is the open question.

Reproducibility statement

All results were produced by a single harness with two backends sharing identical statistics and verdict logic. A one-command launcher prepares the environment, validates the harness and the algebra on the associative-memory backend (a hard gate), and then runs the same pre-specified protocol on a DeltaNet checkpoint. Tables 1–2 were generated by the toy backend over 60 seeds; Tables 3–4 and the sensitivity analyses over 20 seeds. The language-model results (Tables 5–6) were produced on `fla-hub/delta_net-1.3B-100B` in `bfloat16` on one NVIDIA RTX 3090 (`flash-linear-attention` 0.5.1, `transformers` 5.12.1, `torch` 2.12.1/CUDA 13) over 30 seeds. Decoding is greedy and seeds are fixed, so runs are reproducible up to CUDA kernel nondeterminism. The language-model backend reads and writes the per-layer recurrent state through the `flash-linear-attention` cache (in version 0.5.x, `cache[i]` exposes the layer state dict, whose `recurrent_state` we clone, add, truncate and overwrite); this accessor depends on the library’s internal layout and is the single point of adaptation, flagged in the code. Transfer acts on the recurrent state only; a constant skill-agnostic primer supplies the short-convolution window at readout, and the skill table is shown three times to write a usable engram (§6.6). Governance reads the authorized key subspace from the model’s actual keys, captured by a forward hook on each layer’s key short-convolution; this is the second, and only other, point of library coupling. The released artifact includes the exact commands, the complete raw run logs (all 30/30 and 20/20 seeds with final tables), and a per-seed CSV of every condition accuracy emitted by the harness with `--dump`, so that each reported mean and interval is recomputable from the per-seed values; software versions and the base random seed are recorded alongside.

References

- [1] J. Schmidhuber. Learning to control fast-weight memories: an alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- [2] G. E. Hinton and D. C. Plaut. Using fast weights to deblur old memories. In *Proc. Cognitive Science Society*, 177–186, 1987.
- [3] J. Ba, G. E. Hinton, V. Mnih, J. Z. Leibo, and C. Ionescu. Using fast weights to attend to the recent past. In *NeurIPS*, 2016.
- [4] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, PMLR 139:9355–9366, 2021.
- [5] K. Irie and J. Schmidhuber. A modern self-referential weight matrix that learns to modify itself. *arXiv:2202.05780*, 2022.
- [6] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Gated delta networks: improving Mamba2 with the delta rule. In *ICLR*, 2025. *arXiv:2412.06464*.
- [7] B. Peng et al. RWKV-7 “Goose” with expressive dynamic state evolution. *arXiv:2503.14456*, 2025.
- [8] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.
- [9] G. Ilharco, M. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi. Editing models with task arithmetic. In *ICLR*, 2023.

- [10] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: low-rank adaptation of large language models. arXiv:2106.09685, 2021.
- [11] R. Hendel, M. Geva, and A. Globerson. In-context learning creates task vectors. In *Findings of EMNLP*, 2023. arXiv:2310.15916.
- [12] E. Todd, M. L. Li, A. Sen Sharma, A. Mueller, B. C. Wallace, and D. Bau. Function vectors in large language models. In *ICLR*, 2024. arXiv:2310.15213.
- [13] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid. Steering language models with activation engineering. arXiv:2308.10248, 2023.
- [14] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, et al. Representation engineering: a top-down approach to AI transparency. arXiv:2310.01405, 2023.
- [15] S. Liu, H. Ye, L. Xing, and J. Zou. In-context vectors: making in-context learning more effective and controllable through latent space steering. In *ICML*, 2024. arXiv:2311.06668.
- [16] M. Pióro, M. Wołczyk, R. Pascanu, J. von Oswald, and J. Sacramento. State soup: in-context skill learning, retrieval and mixing. arXiv:2406.08423, 2024.
- [17] Y. Jafari, Z. Wang, L. Bergen, and T. Berg-Kirkpatrick. Studying the soupability of documents in state space models. arXiv:2505.24033, 2025.
- [18] M. Belitsky, D. J. Kopiczko, M. Dorkenwald, M. J. Mirza, J. R. Glass, C. G. M. Snoek, and Y. M. Asano. KV cache steering for controlling frozen LLMs. arXiv:2507.08799, 2025.
- [19] H. Ramsauer et al. Hopfield networks is all you need. In *ICLR*, 2021.
- [20] J. Ferber. *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley, 1999.
- [21] P.-P. Grassé. La reconstruction du nid et les coordinations interindividuelles. *Insectes Sociaux*, 6:41–80, 1959.
- [22] B. Spinoza. *Ethica, Part II*. 1677.